

Task 4

1. Introduction

This report outlines the design, implementation, and analysis of a program designed to detect deadlocks across multiple system configurations. The program evaluates process allocations and outstanding resource requests to identify permanently blocked states.

2. Component Analysis

- **Existence Vector (E):** A 1D vector representing the total absolute instances of each resource type present in the system.
- **Allocation Matrix (C):** A 2D matrix tracking the quantity of each resource currently held by each process.
- **Request Matrix (R):** A 2D matrix tracking the outstanding, unfulfilled resource requests needed by each process to complete execution.
- **Available Vector (A):** A dynamically calculated 1D vector representing resources that are currently unallocated and free for use.
- **Finish Vector (Finish):** A boolean vector indicating whether a process can run to completion, initialized to true for processes holding zero resources.

3. Execution Flow

- **Initialization & File I/O:** The program expects the input file path as a command-line argument. It validates the argument count and opens the specified file, first reading the total number of test cases to execute.

```
int main(int argc, char* argv[]) {
    if (argc != 2) {
        cerr << "to use enter the source code followed by the input file name"<< endl;
        return 1;
    }
    string filename = argv[1];
    ifstream infile(filename);
    if (!infile) {
        cerr << "Error: Could not open input.txt\n";
        return 1;
    }

    int n;
    if (!(infile >> n)) {
        cerr << "Error: Could not read the number of test cases.\n";
        return 1;
    }
}
```

- **Test Case Iteration:** An outer loop processes each system scenario sequentially, re-initializing the E, C, R, and A structures for every individual test case.

```
// Initialize vectors with dynamic sizes
vector<int> E(num_resources); //existing recourses in the system
vector<vector<int>> C(num_processes, vector<int>(num_resources)); //the currently held recourses by each process
vector<vector<int>> R(num_processes, vector<int>(num_resources)); //the requested recourses needed by each process

// Read Existence Vector (E)
for (int i = 0; i < num_resources; ++i) {
    infile >> E[i];
}

// Read Current Allocation Matrix (C)
for (int i = 0; i < num_processes; ++i) {
    for (int j = 0; j < num_resources; ++j) {
        infile >> C[i][j];
    }
}
}
```

- **Availability Calculation:** For each scenario, the program computes the initial Available vector (A) by iterating through the Allocation Matrix (C) and subtracting all currently allocated resources from the Existence Vector (E).

```
vector<int> A = E; //the available vector
for (int i = 0; i < num_processes; ++i) {
    for (int j = 0; j < num_resources; ++j) {
        //subtract from the total available
        A[j] -= C[i][j];
    }
}
```

- **Deadlock Detection Loop:** A do-while loop continuously scans for unfinished processes.
 - **State Logging:** The system outputs trace messages (e.g., "Attempting to allocate for process: X") to provide real-time visibility into the algorithm's decision-making.
 - **Failure Logging:** If a process's pending requests (R) exceed available resources (A), the program logs the specific resource index that caused the failure, displaying the required versus available amounts.
 - **Success Logging & Release:** If requests are satisfied, the program simulates process completion. It logs the successful allocation, adds the currently held resources (C) back to the Available vector (A), prints the updated available resource counts, and marks the process as finished.

```

do {
    process_finished = false;
    for (int i = 0; i < num_processes; ++i) {
        if (!Finish[i]) {
            bool can_allocate = true;

            // Check if Request <= Available
            cout << "Attempting to allocate for process: " << i << '\n';
            for (int j = 0; j < num_resources; ++j) {
                if (R[i][j] > A[j]) {
                    can_allocate = false; //made false if on resource cant be allocated
                    cout << "Failed to allocate for resource: " << j << '\n';
                    cout << "Required: " << R[i][j] << " available: " << A[j] << '\n';
                    break;
                }
            }

            //as if the process is completed and the held recourses are released
            if (can_allocate) {
                cout << "-----\n";
                cout << "Successfully allocated and completed for process: " << i << '\n';
                cout << "-----\n";
                for (int j = 0; j < num_resources; ++j) {
                    //add the released to the available
                    A[j] += C[i][j];
                    cout << "New available for resource " << j << " is " << A[j] << '\n';
                }
                Finish[i] = true;
                process_finished = true;
            }
        }
    }
}

```

- **Output Evaluation:** The loop terminates when a full pass yields no new process completions. The system then evaluates the Finish vector, outputting the specific IDs of any deadlocked processes or confirming a deadlock-free state for that test case.

```
bool is_deadlock = false;
vector<int> deadlocked_processes;

for (int i = 0; i < num_processes; ++i) {
    if (!Finish[i]) {
        is_deadlock = true;
        //adding deadlocked processes
        deadlocked_processes.push_back(i);
    }
}

if (is_deadlock) {
    cout << "Deadlock detected in the system.\n";
    cout << "Deadlocked processes: ";
    for (int p : deadlocked_processes) {
        cout << "P" << p << " ";
    }
    cout << "\n";
} else {
    cout << "No deadlock detected in the system.\n";
}
```

4. Conclusion

The code provides a robust and highly observable simulation of a multi-instance deadlock detection algorithm. By introducing test-case iteration and detailed console logging, it offers clear insights into the resource allocation lifecycle, allowing for precise tracking of how and why system deadlocks occur or are avoided across varying scenarios.

5. GitHub: [repo](#)

commit sha: 621074752b4778ca677da9ac93a131009ccb55cc